

Digital Signal Processing Lab

Experiment 2 : RECURSIVE AND ITERATIVE FFT ALGORITHMS
FOR $N=2^m$

Group 06 (WEDNESDAY)



N DEEKSHITHA

24EC10105

SHRAVI BANSAL

24EC10130

Objectives

In this experiment, we need to perform:

- Recursive FFT algorithm
- Iterative FFT algorithm
- Compute the mean - squared error between the output of fft function and output of the Matlab's in - built fft function
- Run - time of DFT formula using brute - force calculations and the recursive and iterative FFT functions

GOAL 1 : Implement the recursive and the iterative FFT algorithms as listed in the manual.

CODE FOR RECURSIVE FFT:

```
close all;
clc;

m = 6;
N = 2^m;

Fs = 12*1e3; % sampling frequency
Ts = 1/Fs;
n = 0:N-1;
x_sampled = 10*cos(2*pi*1e3*n*Ts)+6*cos(2*pi*1e3*2*n*Ts)+2*cos(2*pi*1e3*4*n*Ts);
X_n = x_sampled(1:N);

function [X]=FFT_recursive(X_n,N)
    if N == 1 %Base case
        X=X_n;
        return;
    else
        X_even = FFT_recursive(X_n(1:2:end), N/2); %Calling the function again
        X_odd = FFT_recursive(X_n(2:2:end), N/2);
        wn = exp((-1j)*(2*pi/N));
        W=1;
        X = zeros(1,N);
        for k = 1:N/2
            X(k)= X_even(k)+W*X_odd(k);
            X(k+N/2)= X_even(k)-W*X_odd(k);
            W = W*wn;
        end
        return;
    end
end
```

```

tic;
X = FFT_recursive(X_n,N);
runtime_rec = toc;

figure; %Plot of Recursive FFT and in built matlab fft function
subplot(2,2,1);
plot(abs(X)/N,'b');
title('Recursive FFT plot');
xlabel('Frequency');
ylabel('|X(FFT_Recursive)|/N');
hold on;
Xf = fft(X_n);
subplot(2,2,2);
plot(abs(Xf)/N,'r');
title('In - built Matlab FFT function');
xlabel('Frequency');
ylabel('|X(f)|/N');

figure; %Stem plot of Recursive FFT and in built matlab fft function
f = (-N/2:N/2-1)*Fs/N;
subplot(2,2,1);
stem(f, abs(Xf)/N,'filled');
title('Stem plot of FFT in - built matlab function');
xlabel('Frequency');
ylabel('|X(f)|/N');
subplot(2,2,2);
stem(f,abs(X)/N,'filled');
title('Stem plot of FFT Recursive');
xlabel('Frequency');
ylabel('X(fft_Recursive)/ N');

```

CODE FOR ITERATIVE FFT:

```

function [X]=FFT_Iterative(X_n,m)
    N = 2^m;
    rev=bitrevorder(1:N);

    X=X_n(rev);
    for s=1:m
        M=2^s;
        WM = exp((-1j)*(2*pi)/M);
        for k=1:M:N

```

```

        w=1;
        for p=0:(M/2)-1
            t=w*X(k+p+M/2);
            u=X(k+p);
            X(k+p)=u+t;
            X(k+p+M/2) = u-t;
            w = w*WM;
        end
    end
end
return;
end

tic;
X =FFT_Iterative(X_n,6);
runtime_iter =toc;

figure;          %Plot of Iterative FFT and in built matlab fft function
subplot(2,2,1);
plot(abs(X)/N,'b');
title('Iterative FFT plot');
xlabel('Frequency');
ylabel('|X(FFT_Iterative)|/N');
hold on;
Xf = fft(X_n);
subplot(2,2,2);
plot(abs(Xf)/N,'r');
title('In - built Matlab FFT function');
xlabel('Frequency');
ylabel('|X(f)|/N');

figure;          %Stem plot of Iterative FFT and in built matlab fft function
f = (-N/2:N/2-1)*Fs/N;
subplot(2,2,1);

stem(f, abs(Xf)/N,'filled');
title('Stem plot of FFT in - built matlab function');
xlabel('Frequency');
ylabel('|X(f)|/N');
subplot(2,2,2);
stem(f,abs(X)/N,'filled');
```

```

title('Stem plot of FFT Iterative');
xlabel('Frequency');
ylabel('X(fft_iterative)/ N');

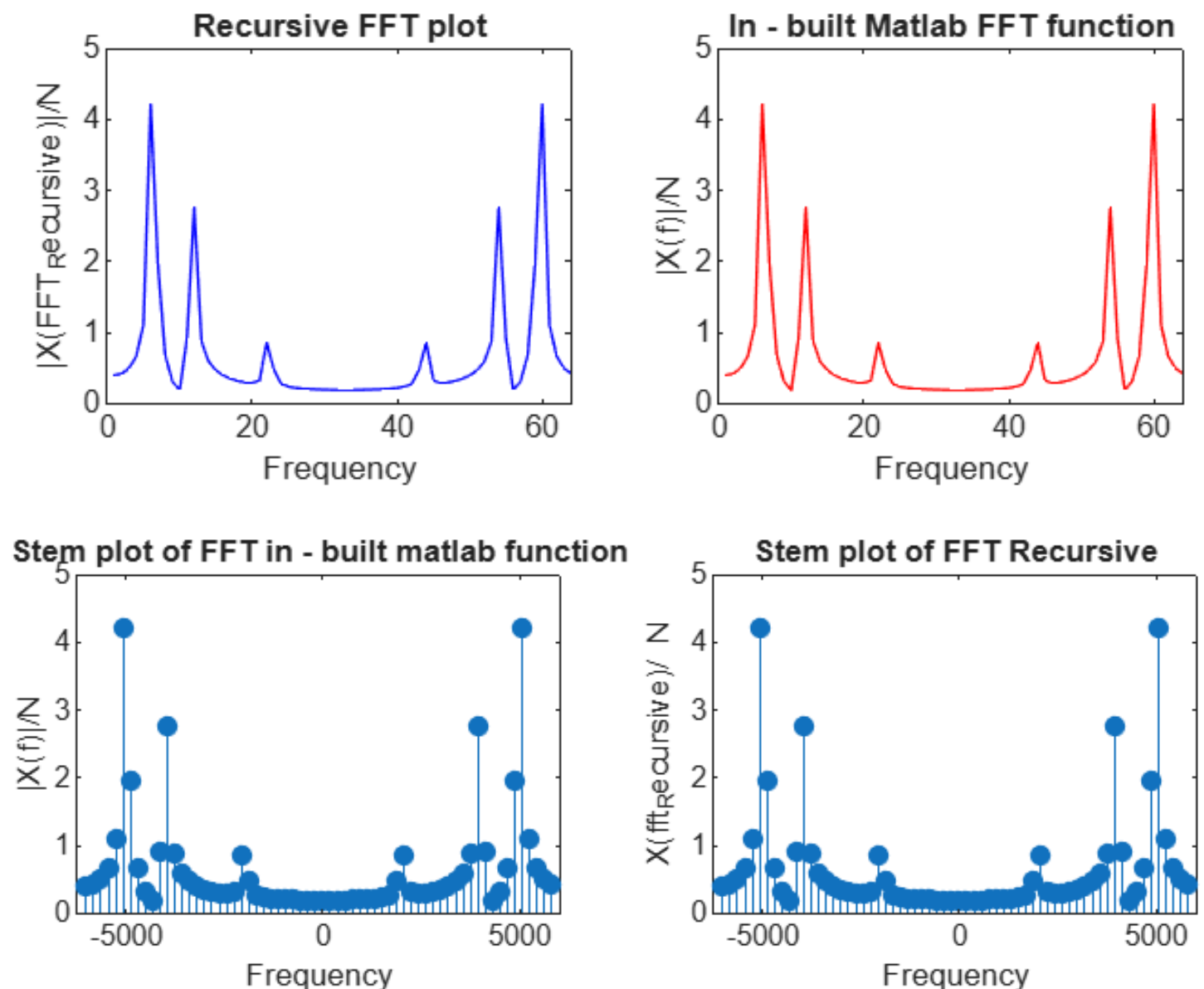
```

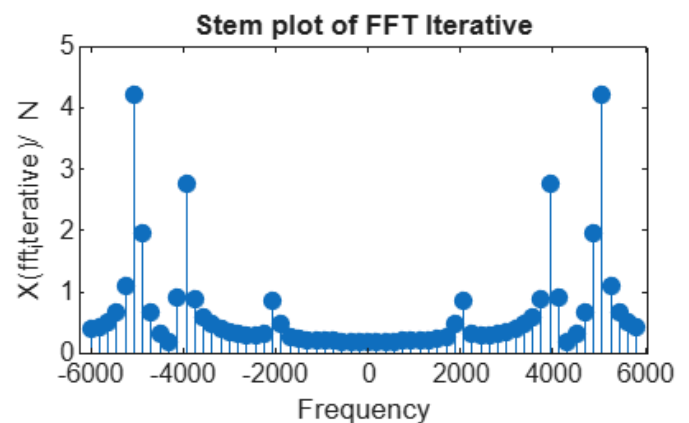
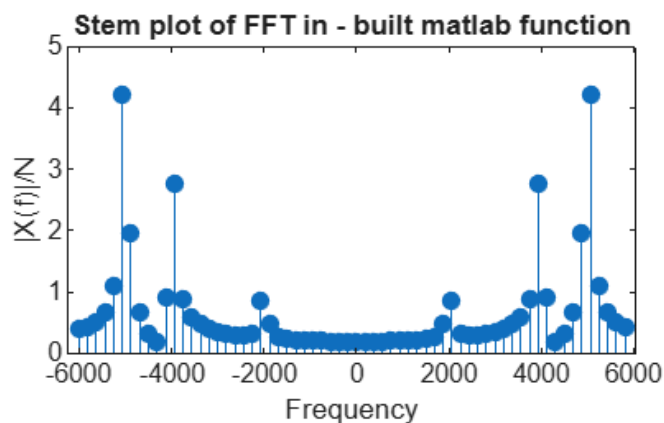
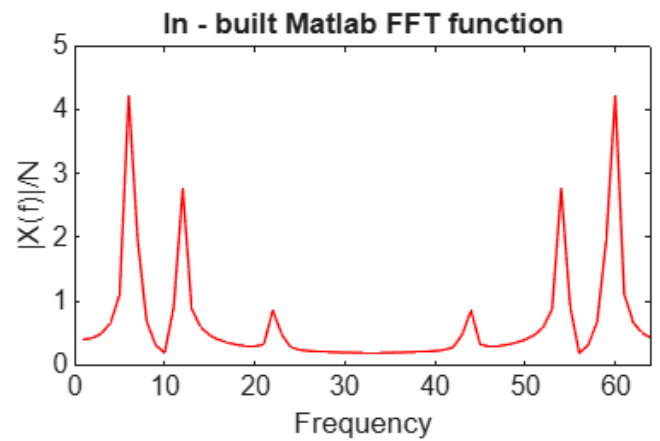
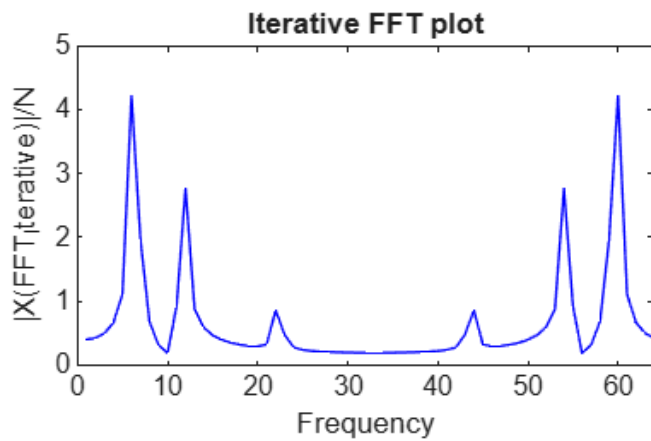
GOAL 2 : Compare the outputs of your implementation (for both recursive and iterative im-plementations) with the output of the Matlab fft function for an $N = 64$ point sequence. If your implementation is correct, the normalized mean-squared error, given by ϵ ,

$$\epsilon = \frac{\sum_0^{N-1} |X[k] - X^*[k]|^2}{\sum_0^{N-1} |X^*[k]|^2}$$

where $\{X[k]\}$ denotes the output of your fft function and $\{X^*[k]\}$ denotes the output of Matlab's in-built fft function.

For $N = 64$ the plots :





CODE FOR MEAN SQUARED ERROR CALCULATION AND PLOT:

```

N = 64;
Fs = 12*1e3; % sampling frequency
Ts = 1/Fs;
n = 0:N-1;
x_sampled = 10*cos(2*pi*1e3*n*Ts)+6*cos(2*pi*1e3*2*n*Ts)+2*cos(2*pi*1e3*4*n*Ts);
X_n = x_sampled(1:N);
w = exp((-1j)*(2*pi/N));

X_rec = FFT_recursive(X_n,N);
X_iter = FFT_Iterative(X_n,6);
X_fft=fft(X_n);

ep_rec = sum( (abs(X_rec - X_fft)).^2 ) / sum( (abs(X_fft)).^2 );
ep_iter = sum( (abs(X_iter - X_fft)).^2 ) / sum( (abs(X_fft)).^2 );

fprintf('Recursive FFT NMSE = %e\n', ep_rec);
fprintf('Iterative FFT NMSE = %e\n', ep_iter);

for k=1:N
    e_rec(k)=(abs(X_rec(k)-X_fft(k)))^2 / (abs(X_fft(k)))^2;

```

```
e_iter(k)=(abs(X_iter(k)-X_fft(k)))^2 / (abs(X_fft(k)))^2;
```

```
end
```

```
figure;
```

```
plot(e_rec,'k');
```

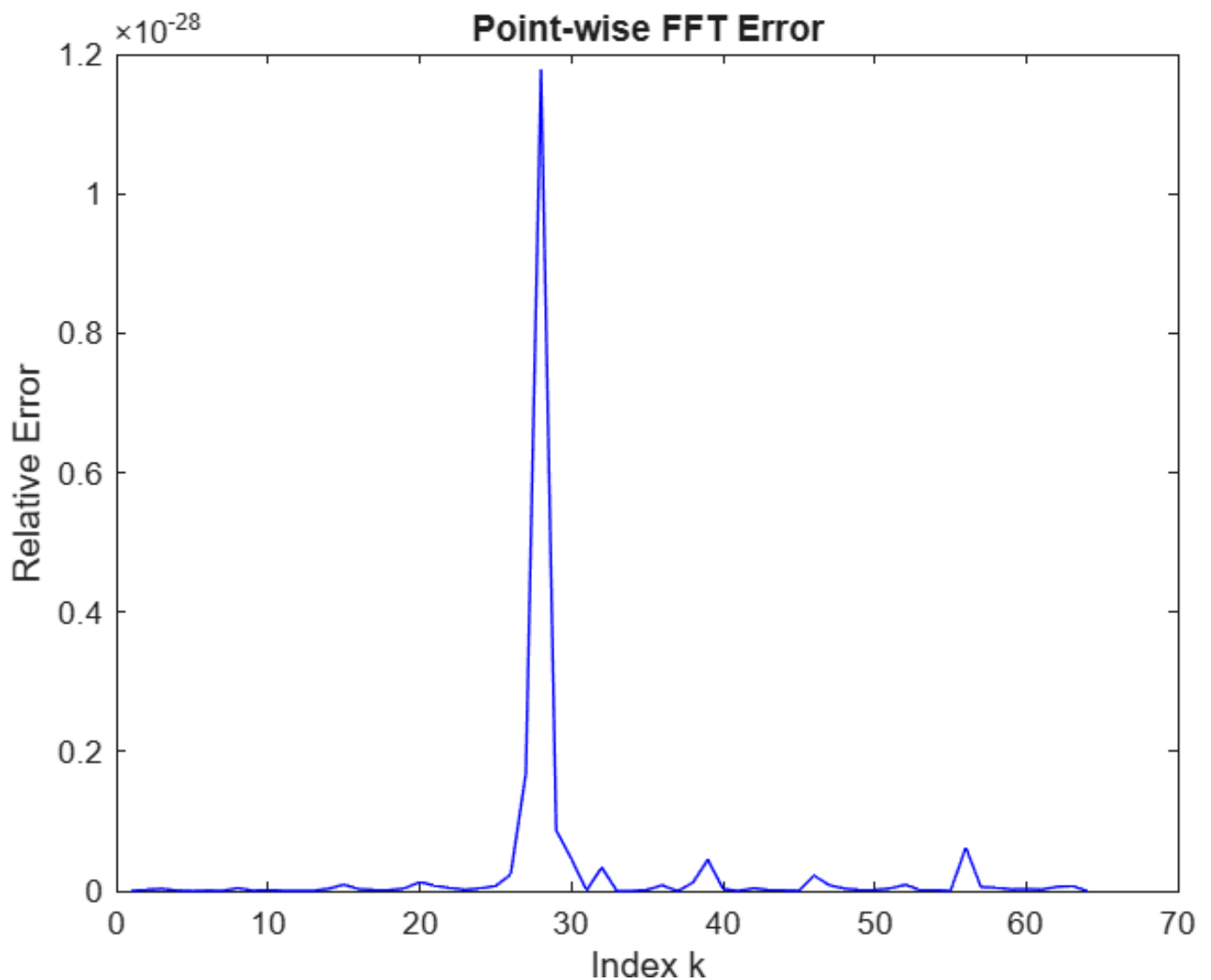
```
hold on;
```

```
plot(e_iter,'b');
```

```
xlabel('Index k');
```

```
ylabel('Relative Error');
```

```
title('Point-wise FFT Error');
```



Recursive FFT NMSE = 2.516145×10^{-31}

Iterative FFT NMSE = 2.516145×10^{-31}

GOAL 3 : Write a function named dft naive that implements the DFT formula using brute-force calculations (i.e., directly from the definition). Plot the run-time of this

function and the recursive and iterative FFT functions that you wrote as a function of N (say, for $N = 2^m$, where $m = 3, 4, \dots, 10$.

CODE FOR DFT NAIVE:

```
function [X]=DFT_naive(X_n)
    N = length(X_n);
    if N == 1
        X=X_n;
        return;
    end
    X = zeros(1,N);
    for k = 0:N-1

        for n=0:N-1
            X(k+1)=X(k+1)+(X_n(n+1)*exp(-2j*pi*k*(n/N)));
        end
    end
    return;
end

tic;
X = DFT_naive(X_n);
runtime_naive = toc;

p=3:10;
runtime_iter = zeros(1, length(p));
runtime_rec = zeros(1, length(p));
runtime_naive = zeros(1, length(p));

for m = 1:length(p)
    N=2^p(m);
    w = exp((-1j)*(2*pi/N));
    Fs = 12*1e3; % sampling frequency
    Ts = 1/Fs;
    n = 0:N-1;
    x_sampled = 10*cos(2*pi*1e3*n*Ts)+6*cos(2*pi*1e3*2*n*Ts)+2*cos(2*pi*1e3*4*n*Ts);
    X_n = x_sampled(1:N);

    tic;
    x_iter= FFT_Iterative(X_n,p(m));
    runtime_iter(m)=toc;
```

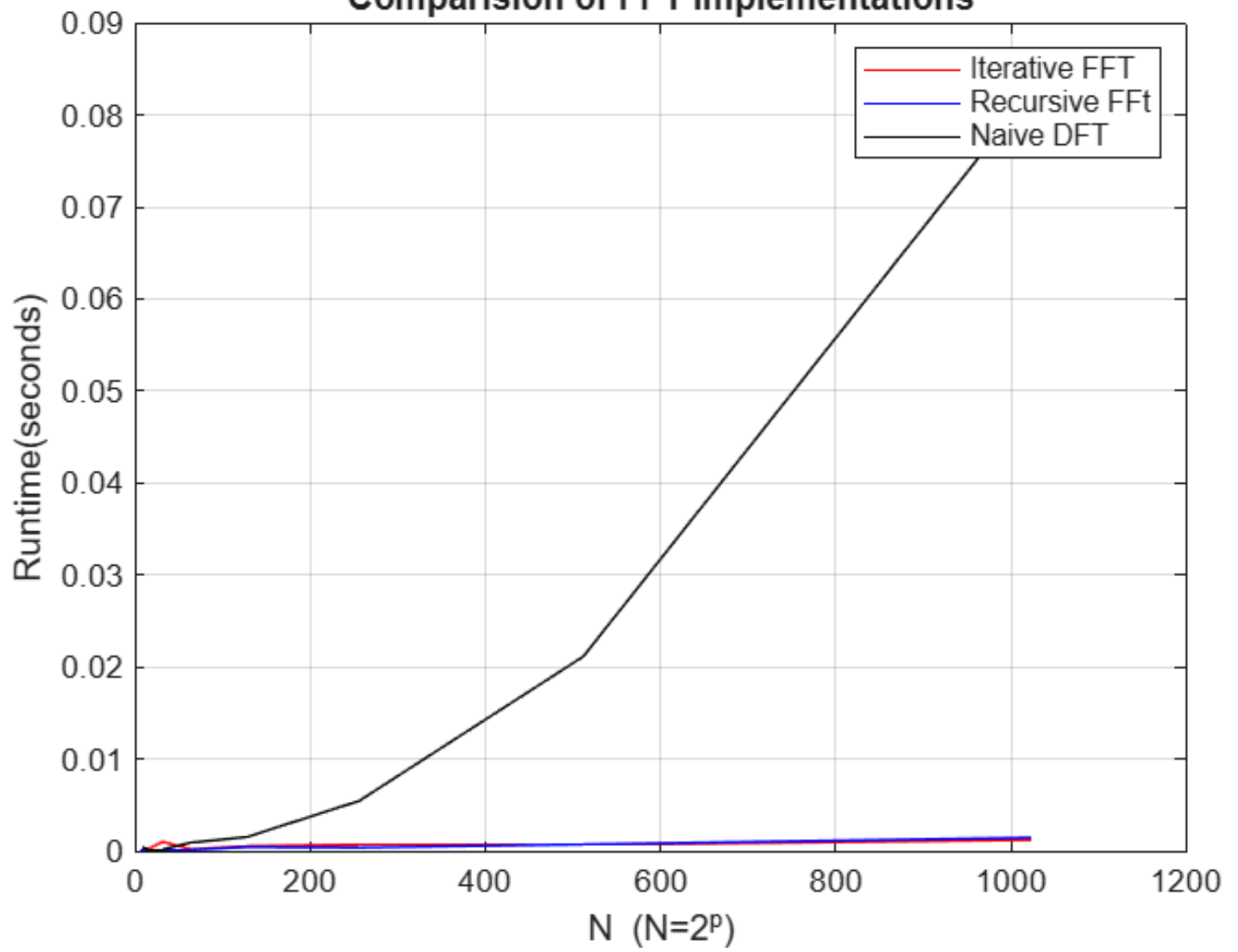
```
tic;
    x_recur= FFT_recursive(X_n,N);
runtime_rec(m)=toc;
tic;
    x_naive= DFT_naive(X_n);
runtime_naive(m)=toc;
end

p=3:10;
N=2.^p;

figure;

plot(N, runtime_iter, 'r');
hold on;
plot(N, runtime_rec, 'b');
plot(N, runtime_naive, 'k');
hold off;
xlabel('N (N=2^p)');
ylabel('Runtime(seconds)');
title('Comparision of FFT Implementations');
legend('Iterative FFT', 'Recursive FFt', 'Naive DFT');
grid on;
```

Comparison of FFT Implementations



RESULTS :

Goal 1 :

1. The core findings from the attached FFT experiment are as follows: both recursive and iterative FFT implementations yield theoretically identical results, with mean squared error nearly zero compared to MATLAB's built-in FFT, and their computational efficiency vastly exceeds the naive DFT, particularly as N increases.

Goal 2 :

1. The output of the recursive FFT and iterative FFT matched the MATLAB built-in FFT for $N = 64$, demonstrating near-perfect agreement on amplitude spectra and stem plots.
2. The iterative FFT typically executes slightly faster than the recursive variant, likely due to reduced overhead from function calls and improved memory access patterns during bit-reversal and butterfly computations.
3. The normalized mean squared error (NMSE) between both custom FFT methods and MATLAB's FFT was calculated as approximately $2.5161e-31$, indicating a negligible numerical discrepancy primarily attributable to machine precision limits rather than algorithmic differences.

Goal 3 :

1. The runtime comparison plot confirms that the naïve DFT's computational time increases quadratically with N , while both FFT approaches scale logarithmically, making them suitable for real-time or large-scale signal processing tasks.
2. The runtime for iterative is comparative to that of the recursive for small N values (less than 1000), but the runtime for naïve is quite very large compared to both, thus
 - a. For $N = 64$ the runtime for recursive is 0.0001, runtime for iterative is 0.0002, runtime for naïve is 0.0010
 - b. For $N = 512$ the runtime for recursive is 0.0007, runtime for iterative is 0.0006, runtime for naïve is 0.0267
 - c. For $N = 1024$ the runtime for recursive is 0.0014, runtime for iterative is 0.0014, runtime for naïve is 0.0961
3. As sample size N increases, FFT implementations (both recursive and iterative) continue to agree in output accuracy and show substantial run-time advantage over naïve DFT, with FFTs executing in milliseconds and DFT requiring seconds for large N .

DISCUSSION :

N DEEKSHITHA (24EC10105) :

- In goal 1:
 - In the recursive function the N value changes as we the factor of 2 as we call the function again and again.
 - We can even compute radix(of any number which can be written as prime factors) not only radix 2 .
 - Iterative FFT requires reordering the input sequence into *bit-reversed order* before performing butterfly operations which is a very mandatory step.
 - Thus, in iterative part $N/2$ butterflies are computed and there $\log_2 N$ total to be computed,

making the complexity $N\log_2 N$.

- As w changes with the change in N , when we call recursive function again in the recursive function we change the w as N changes to $N/2$ it is defined inside the for loop.
- Although both recursive and iterative FFTs achieve the same theoretical complexity of $O(N\log N)$, in practice the iterative version performs better.

- In goal 2:

- The error of both the recursive and iterative are almost same which is also the theoretic result, in this case this error value was found out to be $2.5161e-31$
- As N increases, the both recursive and iterative result is seen to be the same
- The normalized mean square error found out to be same for both the cases, iterative and recursive.
- At $N=64$ almost the recursive, iterative, in-built plots are the coinciding.
- The error is not due to the algorithm but due to finite precision arithmetic.

- In goal 3:

- The time complexity of the naive function $O(N^2)$ is quite more high than compared to that of recursive and iterative FFT $O(N\log_2 N)$.
- As the N increases to 1000 and above the difference between the runtime of the DFT and the FFT becomes almost very large, thus DFT in seconds, whereas FFT in milliseconds (less than that).

SHRAVI BANSAL (24EC10130) :

- GOAL1:

In Recursive FFT:

- based on the Cooley-Tukey approach, recursively splits the input x into even- and odd-indexed elements.

Reason: DFT can be decomposed into DFTs of even and odd parts, reducing the size of each subproblem to half. This exploits symmetry and periodicity by recursively partitioning the DFT formula.

- This follows the divide-and-conquer approach, recursively splitting the input, applying FFT to subarrays, and combining them.
- The twiddle factors $W_{Nk} = e^{-2\pi i k/N}$ rotate and recombine the results, matching the structure of the DFT formula for the original sequence, and ensuring accuracy in the frequency domain representation

In Iterative FFT: (Radix-2 Decimation-in-Time) uses a loop-based approach to accomplish the same decomposition efficiently, eliminating recursion.

- Ensure the input size N is a power of 2 as the Radix-2 FFT algorithm requires the data to be easily split in half repeatedly for maximum efficiency.
- Use bit-reversal which is necessary for the butterfly structure to work correctly.
- “Butterflies” are a graphical way to represent the sum and difference computations that exploit DFT symmetries.

- Twiddle factors are roots of unity used to adjust the phase of odd-index DFT elements when combining with evens; this is essential for reconstructing the original frequency domain correctly, matching the recursion step in the iterative format

Both algorithms reduce computation from $O(N^2)$ (brute-force DFT) to $O(N \log N)$ by systematically reusing calculations.

- **GOAL2:**

- The extremely small, normalized mean-squared error (NMSE) values—about $2.52e-31$ for both recursive and iterative FFT implementations—indicate our custom FFT algorithms are virtually identical to MATLAB's built-in `fft` function for an $N=64$ sequence.
- The only difference arises from very tiny rounding errors inherent in floating-point arithmetic. These are normal and unavoidable in any numerical computation.

NMSE is calculated as:

$$\epsilon = \frac{\sum_{k=0}^{N-1} |X[k] - X^*[k]|^2}{\sum_{k=0}^{N-1} |X^*[k]|^2}$$

- In our case, the numerator (difference between outputs) is extremely small, while the denominator (energy in MATLAB's output) is much larger giving an error value close to machine precision.
- **GOAL3:**
 - The graph clearly shows that the naive DFT implementation has a much higher runtime compared to both the recursive and iterative FFT implementations, especially as the input size N increases.
 - The naive DFT function computes the Discrete Fourier Transform using the definition, which requires $O(N^2)$ operations for an input of size N . This quadratic complexity causes a steep increase in runtime as N gets larger.
 - Both recursive and iterative FFT functions use the Fast Fourier Transform algorithm, which is much more efficient, with a time complexity of $O(N \log N)$.
 - As a result, the lines for FFT methods remain almost flat as N increases, unlike the naive DFT which rises sharply.
 - The recursive and iterative versions perform almost equally well, indicating that the main speedup comes from the FFT algorithm itself rather than the choice of recursion or iteration.
 - This demonstrates why FFT-based algorithms are always preferred for large datasets in practical applications such as signal processing and communications